LAB-2681: Building Quantum-Capable AI Models

Hands-on lab guide

Lucy Xing, lucy.xing@ibm.com
Va Barbosa, va@us.ibm.com
Adarsh Tiwari, adarsh.tiwari1@ibm.com
IBM Research

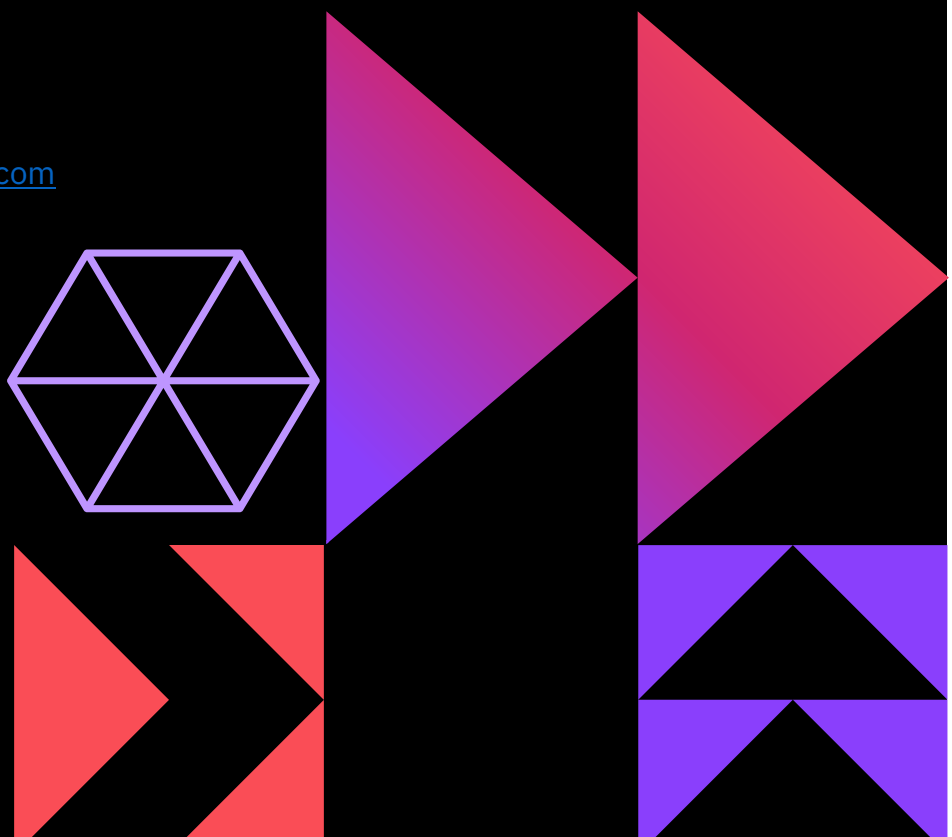


Table of Contents

1 Introduction 5

1.1 About this hands-on lab 6

2 Timing 6

3 Did not complete the lab...?? 6

4 Major task in the lab 6

4.1 Architecture 6

4.2 Environment Setup 7

4.3 Data Generation and Training Observation 7

5 Feedback section 8

6 Getting help and troubleshooting 8

6.1 Common troubleshooting tips ..... **Error! Bookmark not defined.**

6.2 Getting help 8

7 Lab reference materials 8

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed "as is" without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts.
In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply."

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled,
isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. **IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.**

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at: www.ibm.com/legal/copytrade.shtml.

1 Introduction

This lab sits at the intersection of quantum computing and large language models. Writing correct quantum code is hard, and general-purpose LLMs often stumble on it — the domain moves fast and is underrepresented in their training data. In this lab you'll work on both sides of that gap: first using tools to generate quantum-computing training data that could teach a model to do better, then putting models to work on quantum tasks — calling them and running their code against real Qiskit tests so you can verify it works. The tools below are the building blocks you'll use to do that.

Quantum computing is a form of computation that uses the principles of quantum mechanics to process information. Where a classical computer stores information in bits that are either 0 or 1, a quantum computer uses qubits, which can exist in a superposition of both states at once and can be entangled so that the state of one depends on another. Operations are applied through quantum circuits — sequences of gates acting on qubits. In principle, this lets quantum computers explore certain problems (such as simulating molecules, optimization, or cryptography) far more efficiently than classical machines.

This lab uses a handful of tools built around quantum computing and large language models. Qiskit is an open-source Python SDK for quantum computing. It lets you build quantum circuits, simulate them, transpile (optimize and adapt) them for specific hardware, and run them on real quantum computers such as IBM Quantum's systems. It is one of the most widely used toolkits in the quantum software ecosystem.

A large language model (LLM) is an AI system trained on vast amounts of text to predict and generate language. Modern LLMs — such as IBM Granite and Anthropic's Claude — are capable general-purpose reasoning engines: they can write code, explain concepts, and follow multi-step instructions. Their weakness is that their knowledge is fixed at training time, so it can be stale, incomplete, or confidently wrong for fast-moving or specialized domains.

Mellea is IBM's open-source Python library for structured, generative LLM programming. Instead of coaxing a model with one-off, hand-written prompts, Mellea lets you compose generation steps into programs — which makes it well suited to producing synthetic training data at scale. In this lab you'll use it to generate fill-in-the-blank and error-correction data from a set of synthetic prompts.

The common thread across all these tools is verification only recall what quantum code might look like, and that recall is often outdated or subtly wrong. What makes this lab's approach work is that every task comes paired with automated tests built on Qiskit.

Together, these tools form a pipeline for making an LLM more quantum-capable: Qiskit supplies the tasks and verifies the answers, Mellea generates training data at scale, and the result is used to fine-tune a model that writes quantum code better than it did before.

1.1 About this hands-on lab

Part 1 — Generating training data with synthetic prompts and Mellea

Improving what a model knows about a domain starts with the data you teach it on. The first half of the lab turns to that lever: generating high-quality training data for quantum computing.

You will start with the provided synthetic prompt data — a diverse, scalable set of quantum-computing prompts that serve as the input to the rest of the pipeline. From those input prompts, you will use Mellea to produce two kinds of derived training data:

1. **Fill-in-the-blank data** - portions of a quantum program are masked and the model learns to complete them correctly
2. **Error-correction data** - deliberately flawed code are used, and the model learns to recognize and repair mistakes

Mellea makes this generation programmable rather than ad hoc, the same pipeline can be re-run and extended to grow the dataset well beyond what hand-crafting prompts one at a time would allow.

Part 2 — Observe results from a trained model

In this part of the lab, you check whether the training made the model better. Instead of guessing, you let the model answer problems it has never seen before and run the unit tests to count how many it gets right. You compare the original model against the trained one, watch how its score improves, and make sure the gain comes from genuinely learning to write correct quantum code rather than just memorizing answers it saw during training.

2 Timing

Lab timing breakdown:

- Presentation (20 mins)
- Lab (55 mins)

3 Did not complete the lab...??

See section 4 for step-by-step instructions to complete the lab.

4 Major task in the lab

4.1 Architecture

This lab demonstrates how a general-purpose AI model can be made genuinely capable of writing correct quantum code, rather than code that merely looks right.

The lab is structured as a pipeline in which every coding task is paired with an automated test that determines whether a given answer is correct. This test-driven foundation underpins each stage of the workflow:

- **Verifiable tasks** — The pipeline begins with a collection of coding tasks, each accompanied by automated tests that define correct behavior.
- **Practice** — The model attempts each task; its answers are run against the tests, and any failures are fed back so it can try again.
- **Training** — The results of this loop are converted into training material and used to teach the model, with the tests serving as the measure of success.
- **Validation** — Finally, the original and trained models are compared on previously unseen problems to confirm that the model has genuinely improved.

Together, these stages form a pipeline that moves from verifiable tasks, to practice, training, and validation with automated testing tying every stage together.

4.2 Environment Setup

Access URL: https://ibm.biz/~SNqM4MF70

Access Code: techxLAB-2681-3b3b

Download lab starter files from IBM Cloud Object Storage

```
git clone https://github.com/eltonjohnfanboy/LAB-2681-techXchange.git
```

Open `LAB-2681-starter-files` folder using IBM Bob IDE and follow the instructions in the `README.md`

4.3 Data Generation and Training Observation

Open `LAB-2681_quantum_data_pipeline.ipynb` and run the file from top to bottom

5 Feedback section

Use this section to provide feedback on the lab.

6 Getting help and troubleshooting

This section provides information about getting help with your lab and some common troubleshooting topics.

6.1 Getting help

- Go through section 7 for reference materials and documentation
- Reach out to lab instructors

7 Lab reference materials

GitHub

Qiskit: https://github.com/qiskit

Mellea: https://github.com/generative-computing/mellea

Documentation

Qiskit: https://quantum.cloud.ibm.com/docs/en/guides

Mellea: https://docs.mellea.ai/